https://dilliganesh.wordpress.com/2017/07/27/how-to-install-configure-zimbra-high-availability-ha/
Domain: dom.com
Server 1 : ip1.1.1
Server2 : ip2.2.2
Server3 : ip3.3.3

Install zimbra on all 3 servers
==========
(https://computingforgeeks.com/how-to-install-zimbra-collaboration-on-ubuntu/)
The recommended system requirements for a Zimbra server which support up to 50 users are:
4 vCPU or more depending on your available resources
8 GB RAM or more
50 GB available disk space
DNS Server - Dnsmasq or Bind DNS server should be fine

sudo apt update && sudo apt -y full-upgrade
[-f /var/run/reboot-required] && sudo reboot -f

sudo apt install git -y

Bind DNS server
—----------

cd ~/
git clone https://github.com/jmutai/scripts.git
cd scripts/zimbra/
./zimbra_bind_setup_ubuntu.sh (enter the details)

Install dependency packages
sudo apt install git
cd ~
git clone https://github.com/jmutai/scripts.git
cd scripts/zimbra/
./zimbra_bind_setup_ubuntu.sh

Download Zimbra Collaboration Open Source Edition
Ubuntu 20.04
cd ~/
wget
https://files.zimbra.com/downloads/8.8.15_GA/zcs-8.8.15_GA_4179.UBUNTU20_64.20211118033954.tgz

tar xvf zcs-8.8.15_GA_*.tgz
cd zcs*/

Install Zimbra Collaboration on Ubuntu 20.04
sudo ./install.sh
type “Y” to accept license terms and start the installation, except this one
Install zimbra-dnscache [Y] N
Set Admin account password − 7>4
Complete configuration and apply.

$ sudo su - zimbra -c "zmcontrol status"

https://ipaddres_or_hostname:7071

AFTER THAT

– Please change name of each nodes refers into mail.lqs.co.in when installing Zimbra

– Set IP address of each nodes refers into mail.lqs.co.in include DNS and /etc/hosts

Stop Zimbra and DNS services on all nodes (node1 and node2)

1.su – zimbra -c “zmcontrol stop”

2.service named stop

3.chkconfig zimbra off

4.chkconfig named off

ANOTHER BLOG
==========
http://chamilinux.blogspot.com/2018/08/how-to-setup-zimbra-high-availability.html

Requirements
• Two nodes with Ubuntu 16.04 server installed.
• Two network cards installed on each node.
• Additional unpartitioned hard drive installed on each node.

• Non root user with sudo privileges setup on each node.

Getting Started

Before starting, you will need to setup IP address on each node. Use the following IP address on each node:

Node1 :

172.16.0.1 on eth0 and 192.168.0.101 on eth1

Node2 :

172.16.0.2 on eth0 and 192.168.0.102 on eth1

IP 192.168.0.103 will be the high availability IP.

Next, you will also need to setup hostname and hostname resolution on each node. So each node can communicate with each other. On the first Node, open the /etc/hosts file and /etc/hostname file:

sudo nano /etc/hosts

Add the following lines at the end of the file:

172.16.0.1 Node1

172.16.0.2 Node2

sudo nano /etc/hostname

Change the file as shown below:

Node1

Save and close the file when you are finished.

On the second node, open the /etc/hosts file and /etc/hostname file:

sudo nano /etc/hosts

Add the following lines at the end of the file:

172.16.0.1 Node1

172.16.0.2 Node2

sudo nano /etc/hostname

Change the file as shown below:

Node2

Save and close the file when you are finished.

Next, update each node with the latest version with the following command:

sudo apt-get update -y

sudo apt-get upgrade -y

Once your system is updated, restart the system to apply these changes.

Install DRBD and Heartbeat

Next, you will need to install DRBD and Heartbeat on both nodes. By default, both are available in Ubuntu 16.04 default repository. You can install them by just running the following command on both Nodes:

sudo apt-get install drbd8-utils heartbeat -y

Next, start DRBD and Heartbeat service and enable them to start on boot time:

sudo systemctl start drbd

sudo systemctl start heartbeat

systemctl enable drbd

```
systemctl enable heartbeat
```

Configure DRBD and Heartbeat

Next, you will need to setup DRBD device on each Node. Create a single partition on second unpartitioned drive /dev/sdb on each Node.

You can do this by just running the following command on each Node:

```
sudo echo -e 'n\np\n1\n\n\nw' | fdisk /dev/sdb
```

Next, you will need to configure DRBD on both nodes. You can do this by creating /etc/drbd.d/r0.res file on each Node.

```
sudo nano /etc/drbd.d/r0.res
```

Add the following lines:

```
global {
usage-count no;
}
resource r0 {
protocol C;
startup {
degr-wfc-timeout 60;
}
disk {
}
syncer {
rate 100M;
}
net {
cram-hmac-alg sha1;
shared-secret "aBcDeF";
}
on Node1 {
device /dev/drbd0;
disk /dev/sdb1;
address 172.16.0.1:7789;
meta-disk internal;
}
on Node2 {
device /dev/drbd0;
disk /dev/sdb1;
address 172.16.0.2:7789;
meta-disk internal;
}
}
```

Save and close the file when you are finished, then open another configuration file on each Node:

sudo nano /etc/ha.d/ha.cf

Add the following lines:

```
# Check Interval
keepalive 1
# Time before server declared dead
deadtime 10
# Secondary wait delay at boot
initdead 60
# Auto-failback
auto_failback off
# Heartbeat Interface
bcast eth1
# Nodes to monitor
node Node1
node Node2
```

Save and close the file.

Next, open the resource file /etc/ha.d/haresources on each Node:

sudo nano /etc/ha.d/haresources

Add the following lines:

```
Node1 192.168.0.103/24 drbddisk::r0 Filesystem::/dev/drbd0::/opt::ext4::noatime zimbra
```

Here, Node1 is the hostname of your main active node, 192.168.0.103 is the floating point IP address, /OPT is a mount point and /dev/drbd0 is DRBD device.

Next, you will need to need to define and store identical authorization keys on both nodes. You can do this by /etc/ha.d/authkeys file on each Node:

sudo nano /etc/ha.d/authkeys

Add the following lines:

```
auth1
1 sha1 your-secure-password
```

Here, your-secure-password is your secure password. Use the same password on both nodes.

Next, create and start DRBD by running the following command on Node1:

```
sudo drbdadm create-md r0
sudo systemctl restart drbd
sudo drbdadm outdate r0
sudo drbdadm -- --overwrite-data-of-peer primary all
sudo drbdadm primary r0
sudo mkfs.ext4 /dev/drbd0
sudo chmod 600 /etc/ha.d/authkeys
sudo mkdir /OPT
```

Once the DRBD disk is created on Node1, create the DRBD disk on Node2 with the following command:

sudo drbdadm create-md r0
sudo systemctl restart drbd
sudo chmod 600 /etc/ha.d/authkeys
sudo mkdir opt
Now, you can verify the DRBD disk is connected and is properly syncing by running the following command:
sudo cat /proc/drbd
If everything is fine, you should see the following output:
version: 8.4.5 (api:1/proto:86-101)
srcversion: F446E16BFEBS8B115AJB14H
0: cs:SyncSource ro:Primary/Secondary ds:UpToDate/Inconsistent C r-----
ns:210413 nr:0 dw:126413 dr:815311 al:35 bm:0 lo:0 pe:11 ua:0 ap:0 ep:1 wo:f oos:16233752
[>.....................] sync'ed: 3.3% (14752/14350)M
finish: 0:12:23 speed: 12,156 (16,932) K/sec
Next, start heartbeat on both Nodes to enable the failover portion of your setup.
sudo systemctl start heartbeat
Next, verify the mounted DRBD partition with the following command on Node1:
sudo mount | grep drbd
You should see the following output:
/dev/drbd0 on /opt type ext4 (rw,noatime,data=ordered)
Next, verify the floating IP is only bound to Node1 with the following command:
sudo ip addr show | grep 192.168.0.103
You should see the following output:
inet 192.168.0.103/24 brd 192.168.0.255 scope global secondary eth1:0
Install and Configure Zimbra
Once everything is configured properly on both Nodes, it's time to install Zimbra server on both Nodes.
Run the following command on both Nodes to install Zimbra server:
Please refer the
Basic Zimbra Configurations guide
Next, you will need to disable Zimbra service on both Nodes:
sudo systemctl disable zimbra
Here, we will use Node1 as primary and databases on Node2 should be created and populated through synchronization with Node1. So you will need to stop Zimbra service and remove content inside /opt/zimbra on Node2. You can do this with the following command:
sudo systemctl stop zimbra
sudo rm -rf /opt/*
Next, you will need to create a root user for remote management of and access to the databases on the highly available Zimbra instance.
You can do this by running the following command on Node1:

Su zimbra
Zmcontrol start
Initiate Heartbeat for Zimbra Service
Next, you will need to add the zimbra service in your heartbeat instances on both Nodes. You can do this by editing /etc/ha.d/haresources file:
sudo nano /etc/ha.d/haresources
Modify the following lines:
Node1 192.168.0.103/24 drbddisk::r0 Filesystem::/dev/drbd0::/var/lib/mysql::ext4::noatime Zimbra
Save and close the file, when you are finished.
Once the heartbeat is configured, you will need to restart it on both Nodes.
First, restart heartbeat on Node1:
sudo systemctl restart heartbeat
Next, wait for 50 seconds, then restart heartbeat service on Node2:
sudo systemctl restart heartbeat
Test Heartbeat and DRBD
Now, everything is configured properly, it's time to perform a series of tests to verify that heartbeat will actually trigger a transfer from the active server to the passive server when the active server fails in some way.
First, verify that Node1 is the primary drbd node with the following command on Node1:
sudo cat /proc/drbd
You should see the following output:
version: 8.4.5 (api:1/proto:86-101)
srcversion: F446E16BFEBS8B115AJB14H
O cs:Connected ro:Primary/Secondary ds:UpToDate/UpToDate C r-----
ns:22764644 nr:256 dw:529232 dr:22248299 al:111 bm:0 lo:0 pe:0 ua:0 ap:0 ep:1 wo:f oos:0
Next, we will verify that the DRBD disk is mounted with the following command:
sudo mount | grep drbd
/dev/drbd0 on /opt type ext4 (rw,noatime,data=ordered)
Next, verify the Zimbra service with the following command:
Su zimbra
Zmcontrol status
Next, access zimbra server from the remote machine using floating IP :
https://192.168.0.103
Next, restart heartbeat on Node1:
sudo systemctl restart heartbeat
Now, heartbeat will interpret this restart as a failure of Zimbra on Node1 and should trigger failover to make Node2 the primary server.
You can check that DRBD is now treating Node1 as the secondary server with the following command on Node1:
sudo cat /proc/drbd

```
You should see the following output:
version: 8.4.5 (api:1/proto:86-101)
srcversion: F446E16BFEBS8B115AJB14H
0: cs:Connected ro:Secondary/Primary ds:UpToDate/UpToDate C r-----
ns:22764856 nr:388 dw:529576 dr:22248303 al:112 bm:0 lo:0 pe:0 ua:0 ap:0 ep:1 wo:f oos:0
Now, verify that Node2 is the primary drbd node by running the following command on Node2:
sudo cat /proc/drbd
You should see the following output:
version: 8.4.5 (api:1/proto:86-101)
srcversion: F446E16BFEBS8B115AJB14H
0: cs:Connected ro:Primary/Secondary ds:UpToDate/UpToDate C r-----
ns:412 nr:20880892 dw:20881304 dr:11463 al:7 bm:0 lo:0 pe:0 ua:0 ap:0 ep:1 wo:f oos:0
Next, Check to make sure that Zimbra is running on Node2:
Su zimbra
Zmcontrol status
Now, connect to Zimbra server using floating IP on Node2 from remote user.
https://192.168.0.103
Any issue in DRDB please run below command
run below command in node2
drbdadm secondary all
drbdadm disconnect all
drbdadm -- --discard-my-data connect all
Run this command in node1
drbdadm primary all
drbdadm disconnect all
drbdadm connect all
install
tail -f /var/log/ha-debug
/etc/init.d/heartbeat start
tail -f /var/log/ha-debug
df
```

https://www.howtoforge.com/tutorial/how-to-set-up-nginx-high-availability-with-pacemaker-corosync-and-crmsh-on-ubuntu-1604/

Added all server in /etc/hosts

Install nginx

Configure nginx a s in the doc

Install Pacemaker, Corosync, and Crmsh

```
# apt install -y pacemaker corosync crmsh

systemctl stop corosync
systemctl stop pacemaker

apt install -y haveged

corosync-keygen

ls -lah /etc/corosync/

cd /etc/corosync/
mv corosync.conf corosync.conf.bekup
```

Then create a new 'corosync.conf' configuration file with vim.

```
vim corosync.conf
```

Paste the configuration below into that file.

```
# Totem Protocol Configuration
totem {
  version: 2
  cluster_name: hakase-cluster
  transport: udpu
```

```
# Interface configuration for Corosync
  interface {
      ringnumber: 0
      bindnetaddr: 10.0.15.0
      broadcast: yes
      mcastport: 5407
  }
}

# Nodelist - Server List
nodelist {
  node {
      ring0_addr: web01
  }
  node {
      ring0_addr: web02
  }
  node {
      ring0_addr: web03
  }
}

# Quorum configuration
quorum {
  provider: corosync_votequorum
}
```

```
# Corosync Log configuration
logging {
  to_logfile: yes
  logfile: /var/log/corosync/corosync.log
  to_syslog: yes
  timestamp: on
}

service {
  name: pacemaker
  ver: 0
}
```

Save the file and exit the editor.

```
scp /etc/corosync/* root@web02:/etc/corosync/
scp /etc/corosync/* root@web03:/etc/corosync/
```

Start Corosync and add it to start automatically at boot.

```
systemctl start corosync
systemctl enable corosync
```

Now start pacemaker and enable it to start at boot.

```
systemctl start pacemaker
```

```
update-rc.d pacemaker defaults 20 01
systemctl enable pacemaker
```

```
crm status
```

```
corosync-cmapctl | grep members
```

ON MAIN SERVER
==========

Run the crm commands below to disable 'STONITH' and Ignore the Quorum policy.

```
crm configure property stonith-enabled=false
crm configure property no-quorum-policy=ignore
```

Now check STONITH status and the quorum policy with the crm command below.

```
crm configure show
```

Create a new 'virtual_ip' resource for the floating IP configuration with the crm command below.

```
sudo crm configure primitive virtual_ip \
ocf:heartbeat:IPaddr2 params ip="10.0.15.15" \
cidr_netmask="32" op monitor interval="10s" \
meta migration-threshold="10"
```

And for the nginx 'webserver', create the resource with the command below.

```
sudo crm configure primitive webserver \
ocf:heartbeat:nginx configfile=/etc/nginx/nginx.conf \
op start timeout="40s" interval="0" \
op stop timeout="60s" interval="0" \
op monitor interval="10s" timeout="60s" \
meta migration-threshold="10"
```

When this is done, check the new resources 'virtual_ip' and 'webserver' with the command below. Make sure all resources have the status 'started'.

```
crm resource status
```

```
sudo crm configure group hakase_balancing virtual_ip webserver
```

```
crm resource show
```

```
crm status (run this cmd in every server)
```